

# RPi [ Fundamentals & Developments ] with RPi5

## Python Basics

# Thonny IDE is used for Embedded Python Development.

# GPIO zero

## # LED Control through ~~GPIO~~ gpiozero.

from gpiozero import LED. ~~See~~ # importing LED from gpiozero.

~~led = LED(17)~~ # ~~Defin~~

led = LED(17) # initialising the LED.

led.on() # to turn on the led.

led.off() # to turn off the led.

led.toggle() # to toggle (reverse the pin status) of the led

## # Pushbutton Control through gpiozero.

from gpiozero import Button # importing Button from gpiozero.

But = Button(17) # initialising the Button.

callback fn.

Button-status = But.is\_pressed

Button when pressed

returns false if button is not pressed & true if button is pressed (i.e. grounded)



# # PIR with RPi

```
from gpiozero import MotionSensor
```

```
pir = MotionSensor(c4)
```

```
pir.motion_detected
```

its a bool which is 1 when any motion is detected by the PIR sensor.

```
pir.when_motion =   
pir.when_no_motion = 
```

CallBack fncs

```
from signal import pause
```

```
pir.when_motion = led.on
```

```
pir.when_no_motion = led.off
```

#no paranthesis

```
pause()
```

register a callback fn.



# # Using the TERMINAL

pwd # Prints path of current directory.

CD <location> # changes the current dir to desired one.

CD .. # changes the current dir to <sup>its</sup> parent dir.

CD # changes the current dir to root (or /home/user)

ls # displays all the files & sub dirs present inside the current dir.

ls .. # lists all the files & sub dirs present inside the parent dir of the current dir.

ls -a # see all files even if its hidden.

## # Nano

nano <filename>.<sup>extension</sup> # opens the file in nano

touch <filename>.<sup>extension</sup> # creates new file.

rm <filename> # removes file

cat <filename> # to view the file.

• nano.rc → # Config file for nano  
Should be in root directory.



mkdir # creates directory

mkdir ~~new~~ directory\_name. # creates directory

mv target\_path destination\_path # moves a file or folder

cp target\_path destination\_path # copies a file or folder

# IP Address

hostname -I

# df -h

to view the space left in the disk.

# Config

sudo raspi-config

# Shutdown

sudo shutdown now

sudo reboot.



# # file operations in Python.

```
with open ("Path-to-the-file", "mode") as f:  
    for line in f:  
        print(f.read(c)) # use this to  
                           Access the doc line  
                           by line.  
        print(f.read(c))  
    f.write
```

↓

- r # reading
- w # writing
- wt # read-write
- a # append

# in "w" mode, if the file doesn't exist it will be created automatically.

## # OS Module - import os

path & folder name

os.mkdir() # to create a direct.

os.path.exists("path") # returns true if path exists

os.remove("path") # removes file or directory stated in path.

## # Accessing the RPi - Camera from terminal

rpivid - still # Preview the Camera.

rpivid - still -o camera/test.jpg # Capturing a photo  
↓  
Path

--width 1536 } to set Resolution.  
--height 864 }  
-vflip --hflip

# Attributes for horizontal & vertical flipping the photo.



~~picam - vid path~~

~~picam = v~~

~~picam - v - o path/video.mp4 - t 7000~~

~~7000~~

time of video  
(in ms)

# Accessing Camera # With Python.

picamzero → Module for picam

cam = Camera()

or

cam = picamzero.Camera()

} Initializing the  
Camera object

# it's an attribute (no paranthesis)

cam.still\_size = (<width>, <height>)

# setting height & width for  
the camera snapshot

cam.flip camera (hflip = True, vflip = False)

for setting h & v flips  
of image.



```
cam.take_photo("Path/photo.jpg")
```

# to take photo and save it  
to the ~~desired~~ specific  
~~Path~~ Path

```
cam.video_size = (width, height)
```

# setting size for video

```
cam.record_video("path/vid.mp4", 7)
```

time in  
seconds

# to ~~take~~ record video

```
cam.capture_sequence("path/img.jpg", num_images=10,  
interval=5)
```

interval  $\leftarrow$   
b/w two captures

no of images

# for Capturing Sequence of images

## # Web server on RPi Using Flask

importing library  $\rightarrow$  defining object  $\rightarrow$  setting rout

```
from flask import Flask
```

running the  
app on a specific  
Host

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def index():
```

```
    return "Hello Here is your Html code"
```

```
app.run(host="0.0.0.0") default port = 5000
```

```
app.run(host="0.0.0.0" port=3000)
```

Port can also be set via this

## # Connecting Flask With GPIOs

it is done with making <sup>a separate</sup> ~~new~~ route for it.

```
@app.route("/push-button")
```

```
def push_btn():
```

```
    return str(button.is_pressed)
```

## # Launching Programs on boot - Using systemd

Steps

① run `cd /lib/systemd/system`

& services

# in this there are all the ~~files~~ <sup>services</sup> which starts automatically @ startup in back ground.

② create new service.

`sudo nano filename.service`

or

`sudo touch filename.service`

③ In the file

PTO →



[Unit]

Description = ~~Any~~ Any description you want it's not going to be used by it, it's just for your understanding.

After = multi-user.target

[Service]

WorkingDirectory = /Path/ to /the /working /dir

ExecStart = <sup>absolute</sup>Command which has to be executed.

User = ~~p~~ username.

# to find absolute path of ~~any~~ any Command  
run:-  
which <any-command>

[Install]

WantedBy = multi-user.target

# then start the ~~serv~~ service.

sudo systemctl start project-take-photos.service.

sudo systemctl start service-filename.service.

# it will start in Background.

# to view the error log.

journalctl -u ~~project-take-photos~~ service-filename.service.

# to stop the services.

sudo systemctl stop servicename.service.

# to view startup services.

```
sudo systemctl list-unit-files
```

# to enable it so that it can Run Automatically after <sup>from</sup> Next Boot onwards.

```
sudo systemctl enable servicename.service
```

# to disable them

```
sudo systemctl disable servicename.service
```

## # Steps

- ① ~~to~~ go to /lib/systemd/system
- ② Create new service (servicename.service)
- ③ Edit the file as stated Previously.
- ④ start the service (mentioned b4)
- ⑤ view error log & resolve it (optional, if any)
- ⑥ enable the service for running & automatically after boot.